

Laboration 2 – TCP, UDP och observerbar nätverkstrafik

Uppgift

Bygg två små kommunikationsflöden för sensordata: ett med TCP och ett med UDP. Observera trafiken i Wireshark, testa fel och jämför resultaten med MQTT-flödet från dag 1.

Utgå från de plattformsoberoende C++-exemplen i lärardemonstrationen. Samma server- och klientkod kan kompileras med Winsock i Windows och POSIX-sockets i Linux genom den gemensamma hjälpheadern. Funktion och dokumentation är viktigare än kodmängd. En separat Python-version finns endast som alternativ referens.

Krav

- Arbeta i samma grupp och repository som dag 1.
- Använd endast egen eller uttryckligen tillåten labbtrafik i Wireshark.
- Skicka giltig JSON med samma `sensorId`, `value` och `unit` som i caset.
- Använd olika portar för TCP och UDP.
- Dokumentera adresser, portar, protokoll och programmens roller.
- Testa och dokumentera minst ett kommunikationsfel.
- Committa kod och dokumentation utan hemligheter eller paketfångster med olämplig data.

Del 1 – Förbered projektet

1. Hämta senaste versionen av gruppens repository.
2. Skapa en branch, exempelvis `day-2-sockets`.
3. Skapa en katalog för dagens kod, exempelvis `src/day_2/`.
4. Bestäm vem som börjar med server/mottagare, klient/sändare och dokumentation.

Del 2 – TCP-flöde

Skapa eller anpassa en TCP-server som:

- binder till en dokumenterad IP-adress och port,
- lyssnar och accepterar minst en klient,
- tar emot ett JSON-meddelande,
- skriver klientadress och mottagen data i terminalen,
- skickar ett enkelt svar, exempelvis `{"status": "ok"}`.

Skapa eller anpassa en TCP-klient som:

- ansluter till serverns adress och port,
- skickar gruppens JSON-meddelande,
- tar emot och skriver ut serverns svar.

Kör först lokalt med `127.0.0.1`. Om läraren tillåter det kan klient och server därefter köras på olika datorer.

Del 3 – Observera TCP i Wireshark

1. Starta paketfångst på rätt nätverksgränssnitt.
2. Använd filtret `tcp.port == <TCP_PORT>`.
3. Starta servern och kör klienten.
4. Identifiera klientens och serverns IP-adresser och portar.
5. Leta efter anslutningens start och avslut.
6. Kontrollera om JSON-innehållet går att se i den okrypterade testtrafiken.

Spara inte en stor generell paketfångst i repositoryt. Dokumentera hellre relevanta observationer och, om läraren vill, en beskuren skärmbild utan annan trafik.

Del 4 – UDP-flöde

Skapa eller anpassa en UDP-mottagare som:

- binder till en annan dokumenterad port,
- tar emot minst ett datagram,
- skriver sändarens adress och mottagen JSON.

Skapa eller anpassa en UDP-sändare som skickar samma JSON-data till mottagaren.

Fånga trafiken med:

```
udp.port == <UDP_PORT>
```

Jämför hur UDP-trafiken ser ut mot TCP-trafiken.

Del 5 – Inför och analysera fel

Testa minst ett av följande:

- TCP-klienten använder fel port.
- TCP-servern är inte startad.
- UDP-sändaren skickar innan mottagaren startat.
- Mottagaren lyssnar på en annan adress eller port.
- JSON-meddelandet är ofullständigt eller ogiltigt.

Dokumentera:

1. Vad ni ändrade.
2. Vad klient, server, sändare eller mottagare visade.
3. Vad Wireshark visade.
4. Hur ni hittade orsaken.
5. Hur ni verifierade att felet var åtgärdat.

Del 6 – Jämför modellerna

Lägg till en kort jämförelse i `README.md` eller `docs/day_2.md`:

Fråga	TCP	UDP	MQTT
Behövs en anslutning?			
Bevaras meddelandegränser?			
Finns leveransgaranti i protokollet?			
Känner sändaren till mottagaren direkt?			
Vad händer om central tjänst/broker stoppas?			
När passar detta i ert IoT-case?			

Beskriv också med egna ord skillnaden mellan client-server och peer-to-peer. Ange en möjlighet och en begränsning med vardera modellen.

Leverabler före 15:30

- Källkod för fungerande TCP-server och TCP-klient.
- Källkod för fungerande UDP-mottagare och UDP-sändare.
- Dokumenterade adresser, portar och startinstruktioner.
- Wireshark-observationer för TCP och UDP.
- Analys av minst ett infört fel.
- Jämförelse mellan TCP, UDP, MQTT, client-server och peer-to-peer.
- Minst en beskrivande commit för dagens arbete.

Fördjupning om ni blir klara

Välj minst en:

- Låt TCP-servern hantera flera JSON-meddelanden med radslut eller längdprefix.
- Lägg till timeout och kontrollerad återanslutning i TCP-klienten.
- Lägg till sekvensnummer i UDP-meddelanden och upptäck saknade nummer.
- Bygg två UDP-peers som både kan sända och ta emot.
- Mät och jämför antal byte för ett litet testmeddelande utan att dra generella prestandaslutsatser.